Zero-Copy Wire Formats and the End of the Serialization Tax: A Latency Model and Benchmark Methodology for the ZAP Protocol

ZAP Protocol Authors
zap-proto.io

2026

Abstract

Modern microservice deployments spend a measurable fraction of CPU on encoding and decoding RPC payloads. Profiling of warehouse-scale workloads attributes 4–7% of cycles to protocol-buffer-style serialization in hot paths [1]. We present a formal latency model and a benchmark methodology for the ZAP wire format, a Cap'n Proto–derived encoding that places no parsing step between received bytes and program-visible objects. We decompose RPC latency into serialization, transit, deserialization, and handler components, and show that for ZAP the serialization and deserialization terms collapse to constant time in field count, leaving only payload-byte cost. We give a throughput-bound theorem proving that ZAP's effective bandwidth approaches link rate under bounded payload size, while protobuf-over-HTTP/2 plateaus at roughly 60–70% due to HPACK and tag-length-value overhead. We describe a reproducible single-machine and 10 GbE benchmark methodology that reports $p50/p95/p99/p999$ tails under both quiescent and congested regimes. We discuss honestly when ZAP does *not* win, including single-field tiny messages and embedded environments without zero-copy mmap.

1 Introduction

The serialization tax is the cost a service pays, measured in CPU cycles and tail latency, to translate an in-memory object graph into a sequence of bytes for transit and back. Profiling of large fleets shows this is not a rounding error: in a warehouse-scale measurement, serialization libraries together account for 4–7% of all cycles [1]. At the 99.9th percentile, where users notice, the cost is more pronounced because allocator pressure, copy-induced cache misses, and HPACK dynamic-table churn each contribute their own tail.

We argue, and prove, that this tax is not fundamental. It is an artifact of designing wire formats around streaming parsers when the deployment reality is mmap'd memory, persistent connections, and schemas known at compile time. The ZAP wire format is derived from Cap'n Proto [2, 3]: messages are arenas of fixed-layout structs connected by relative pointers, and parsing collapses to the identity function on the byte buffer.

This paper makes three contributions:

1. A formal RAM-model latency decomposition that separates per-field cost from per-byte cost.
2. A throughput bound (Theorem 2) and a tail-latency independence result (Theorem 3) for the ZAP wire format.
3. A benchmark methodology covering small/medium/large RPC sizes, loopback and 10 GbE, post-quantum handshake amortization, and honest discussion of regimes where ZAP loses.

2 Wire Format Recap

2.1 Arenas and segments

A ZAP message is a sequence of *segments*, each a contiguous byte range obtained from an arena allocator. Segments are 8-byte aligned. Multiple segments form a single logical message and may be transmitted back-to-back, mmap'd from a file, or constructed in shared memory.

2.2 Far-pointer model

Pointers are 64-bit relative offsets, not virtual addresses. A pointer encodes target segment id and word offset within the segment, plus a small tag distinguishing struct, list, far, and capability pointers. Because offsets are relative, no fixup is required when the message is copied, mmap'd at a different address, or cast directly out of a network buffer.

2.3 Capability tables vs schema-on-the-wire

Unlike formats that embed field tags in every record (Protocol Buffers [4], MessagePack [5]), ZAP places schema information in compile-time-generated accessors. The on-wire record is purely positional: byte k of a struct of type T is field $T.f_k$ by construction. This is the same trade-off as FlatBuffers [6] but with the addition of mutable arenas and capability references.

3 Formal Latency Model

Definition 1 (RPC latency decomposition)**.** Let an RPC have request size n bytes spanning f logical fields, and let the network round-trip time be ρ and the link bandwidth B. We decompose end-to-end latency as

$$L \;=\; L_{ser} \;+\; L_{net} \;+\; L_{de} \;+\; L_{hdl},$$

with transit cost $L_{net} \;\geq\; \rho + n/B$.

3.1 Per-format costs

Under the unit-cost RAM model with constant cache latency c, we have the following bounds (justified in detail in the companion proof document):

Format	L_{ser}	L_{de}
JSON over HTTP/2	$\Theta(n + f \log f)$	$\Theta(n)$
Protobuf over HTTP/2	$\Theta(f + n)$	$\Theta(f + n)$
Protobuf over QUIC	$\Theta(f + n)$	$\Theta(f + n)$
ZAP (zero-copy)	$\Theta(1) + \Theta(p)$	$\Theta(1)$

where p is the number of variable-length payload bytes that must be *copied into* the arena. For fixed-layout structs $p = 0$ and $L_{ser} = O(1)$.

Remark. The JSON $f \log f$ term arises from key sorting in canonical JSON. The Protobuf f term is varint tag decoding plus length-prefixed dispatch. ZAP has neither because field positions are determined at compile time.

4 Throughput and Tail-Latency Theorems

We state results here; full proofs appear in the companion proof document at `proofs/zero-copy-throughput/proof`

Theorem 2 (Throughput bound for zero-copy wire)**.** *Let a workload offer requests of size n bytes at offered load λ, with link bandwidth B and per-request fixed CPU cost σ (parsing, framing, dispatch). Then the achievable throughput T for a wire format with serialization cost $L_{ser}(n, f)$ and deserialization cost $L_{de}(n, f)$ satisfies*

$$T \;\le\; \min\Big(\tfrac{B}{n+h},\; \tfrac{1}{\sigma + L_{ser}(n,f) + L_{de}(n,f)}\Big),$$

where h is the per-message header overhead. For ZAP under fixed layout, $L_{ser} = L_{de} = O(1)$ and h is $O(1)$ words, so $T \to B/n$ as $n \to \infty$. For Protobuf-over-HTTP/2, $h \ge h_{HPACK} + h_{frame}$ and the CPU bound binds first at small n, capping T at 0.6–$0.7\,B/n$.

Theorem 3 (Tail-latency independence from field count)**.** *Let $L^{(q)}$ denote the q-th quantile of single-RPC latency under i.i.d. payloads with f fields and bounded payload bytes p. For ZAP, $L^{(q)}(f, p) = L^{(q)}(p) + O(1)$, i.e. tail latency is asymptotically independent of field count. For Protobuf and JSON, $L^{(q)}$ grows linearly in f, with the slope at the 99.9th percentile up to $5\times$ the median slope due to allocator and HPACK dynamic-table tail behavior [7, 8].*

5 Benchmark Methodology

5.1 Hardware and software baseline

- CPU: commodity 8-core x86_64 at 3.6 GHz, AVX2, SMT off.
- Memory: 64 GiB DDR4-3200, 2 channels, NUMA pinned.
- NIC: 10 GbE, MTU 9000, RX/TX coalescing tuned for tail.
- OS: Linux 6.x with `tickless` and `nohz_full` on benchmark cores; turbo and C-states disabled.
- TLS / KEM: X25519 plus a post-quantum KEM, handshake amortized over 10^5 RPCs per connection.

5.2 Workloads

Small 16–128 byte payloads, 4–16 fields. Models cache lookups and feature-flag fetches.

Medium 1–16 KiB, 32–256 fields. Models row reads and structured event records.

Large 64 KiB–1 MiB, lists of structs. Models bulk fetches.

Streaming continuous server-push at 100 k msg/s sustained for 5 minutes; measures GC and allocator pressure.

5.3 Measurement protocol

1. Warm-up: 30 seconds at target load, results discarded.
2. Measurement window: 300 seconds.
3. Per-RPC timestamps captured with `rdtsc` (calibrated) on issuer thread; clock-skew corrected per connection.
4. Report median, $p95$, $p99$, $p999$, and max.
5. Repeat 5 runs; report median across runs and 95% CI.
6. Congestion test: inject competing 5 Gbps iperf flow on the same NIC; remeasure tails.

5.4 Compared protocols

gRPC over HTTP/2 with Protobuf [9, 10], raw HTTP/2 with JSON, Protobuf over QUIC [11], and native ZAP. Each implementation is the canonical open-source reference and is run with its own client. Allocators are pinned (`jemalloc`) for fairness.

6 Predicted Results

Table 1 summarizes the headline numbers. Speedups are ratios of the comparator's $p99$ latency to ZAP's $p99$ latency. Numbers are calibrated against published Cap'n Proto vs Protobuf ratios [12] and warehouse-scale tail data [8].

Table 1: Predicted $p99$ latency speedup of ZAP over comparators.

Workload	vs JSON/H2	vs gRPC/H2	vs PB/QUIC	ZAP $p99$ (μs)
Small (64 B, 8 fields)	9–14×	3.5–6×	2.5–4×	8 ± 1
Medium (4 KiB, 64 f.)	7–11×	4–7×	3–5×	22 ± 3
Large (256 KiB)	3–5×	1.6–2.2×	1.3–1.7×	310 ± 15
Streaming 100 k msg/s	6–9×	3–4×	2.5–3.5×	—

6.1 Tail behavior under congestion

We expect Protobuf/HTTP-2 $p999$ to inflate by 4–8× under the competing iperf flow due to HPACK dynamic-table eviction and HOL blocking. ZAP over QUIC streams should inflate by 1.5–2.5×.

7 When ZAP Does Not Win

We are obliged to report negative regimes:

- **Tiny single-field messages.** A 1-field 8-byte payload is dominated by transport headers; the per-message constant in ZAP (segment header, root pointer) costs about 16 bytes that Protobuf can compress with HPACK if reused.

- **Environments without zero-copy mmap.** On platforms that cannot expose contiguous arenas (some embedded RTOSes, certain WebAssembly runtimes pre-memory64), ZAP loses its main advantage and falls back to a conventional copy.
- **Highly heterogeneous schemas.** If consumers share the wire but not the schema, ZAP's positional layout is fragile across versions; protobuf's tag-based skip-on-unknown is easier to live with operationally.
- **Small-bandwidth high-RTT links.** When L_{net} dominates ($\rho \gg L_{ser} + L_{de}$), wire format choice is nearly invisible.

8 Discussion: PQ Handshake Amortization

A post-quantum KEM handshake (X25519 hybridized with ML-KEM-768) costs roughly 1–2 ms once. Amortized over a connection that serves 10^5 RPCs, the per-RPC overhead is 10–20 ns, well below ZAP's per-RPC fixed cost. Long-lived ZAP connections therefore eat the PQ handshake exactly once, in contrast to short-lived gRPC connections that may pay it on every TLS resume failure.

9 Related Work

Cap'n Proto [2] originated the zero-copy arena design. FlatBuffers [6] adopted the same idea with a read-only emphasis. Protocol Buffers [4] and Avro [13] pay tag-length-value cost in exchange for schema-evolution flexibility. MessagePack [5] is JSON-like but binary, with similar parsing cost. HTTP/2 [10] and QUIC [11] attack the transport layer but leave the encoding layer to the application. FaSST [14] and Snap [15] demonstrate that, with kernel-bypass and careful encoding, RPC latency can reach single-digit microseconds; our work targets the encoding contribution to that budget. The classical parser-cost analysis of Aho and Ullman [16] underpins our serialization-cost lower bounds. The "numbers every programmer should know" list [17] calibrates our model's constants.

10 Conclusion

The serialization tax is real, measurable, and avoidable. ZAP's zero-copy wire format collapses serialization and deserialization to constant time in field count, which translates to predictable tail latency and near-link-rate throughput. We have presented a formal model, two theorems with proofs in a companion document, and a reproducible benchmark methodology. The remaining work is empirical: running these benchmarks across a representative hardware matrix and publishing the raw distributions.

References

[1] Svilen Kanev et al. "Profiling a Warehouse-Scale Computer". In: *ISCA*. 2015.

[2] Kenton Varda. *Cap'n Proto: Insanely Fast Data Interchange Format*. `https://capnproto.org/`. Project documentation and encoding specification. 2013.

[3] Kenton Varda. *Cap'n Proto Encoding Specification*. `https://capnproto.org/encoding.html`. 2013.

[4] Google. *Protocol Buffers*. https://protobuf.dev/. 2008.

[5] Sadayuki Furuhashi. *MessagePack: It's like JSON but fast and small*. https://msgpack.org/. 2008.

[6] Google. *FlatBuffers: Memory Efficient Serialization Library*. https://google.github.io/flatbuffers/. 2014.

[7] Jeffrey Dean and Luiz André Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80.

[8] Yiwen Xu et al. "Microsecond-Scale Tail Latency at Scale". In: *OSDI*. 2024.

[9] Google. *gRPC: A High Performance, Open-Source Universal RPC Framework*. https://grpc.io/. 2016.

[10] M. Thomson and C. Benfield. *HTTP/2*. RFC 9113, IETF. 2022. URL: https://www.rfc-editor.org/rfc/rfc9113.

[11] J. Iyengar and M. Thomson. *QUIC: A UDP-Based Multiplexed and Secure Transport*. RFC 9000, IETF. 2021. URL: https://www.rfc-editor.org/rfc/rfc9000.

[12] Kenton Varda. *Cap'n Proto vs Protobuf Benchmarks*. https://capnproto.org/news/2014-06-17-capnproto-flatbuffers-sbe.html. 2014.

[13] Apache Software Foundation. *Apache Avro*. https://avro.apache.org/. 2009.

[14] Anuj Kalia, Michael Kaminsky, and David G. Andersen. "FaSST: Fast, Scalable and Simple Distributed Transactions with Two-Sided RDMA Datagram RPCs". In: *OSDI*. 2016.

[15] Michael Marty et al. "Snap: A Microkernel Approach to Host Networking". In: *SOSP*. 2019.

[16] Alfred V. Aho and Jeffrey D. Ullman. *The Theory of Parsing, Translation, and Compiling*. Prentice-Hall, 1972.

[17] Jeff Dean. *Numbers Every Programmer Should Know*. Stanford CS295 Lecture, also in *Designs, Lessons and Advice from Building Large Distributed Systems*. 2009.